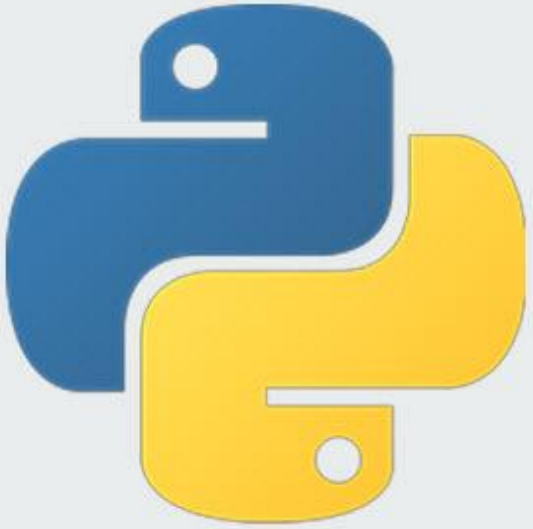
Introducción a Python



Departamento de Informática

Presentación

Python es un lenguaje de programación potente y fácil de aprender. Posee eficientes estructuras de datos de alto nivel y un enfoque sencillo pero efectivo de la programación orientada a objetos. La sintaxis elegante, los tipos de datos dinámicos y el hecho de ser un lenguaje interpretado hacen de Python un lenguaje ideal para la creación de scripts y el desarrollo rápido de aplicaciones en todo tipo de áreas y plataformas.

El intérprete de Python y la biblioteca estándar están disponibles en forma de código fuente o código ejecutable para los principales sistemas operativos en el sitio web oficial de Python, https://www.python.org/.

El intérprete de Python puede ampliarse fácilmente con nuevas funciones y tipos de datos implementados en C o C++ (u otros lenguajes accesibles desde C). Python es también adecuado como lenguaje de extensión de aplicaciones.

Instalación del intérprete e IDE

Windows:

- https://www.python.org/downloads/release/python-379/

Mac:

- brew install python / https://www.python.org/downloads/release/python-379/

Linux:

- apt install python

IDE:

- https://www.jetbrains.com/pycharm/download/ (versión comunity)

Ejemplo práctico

```
edad = int(input("¿Cuántos años tiene? "))
if edad < 18:
    print("Es usted menor de edad")
else:
    print("Es usted mayor de edad")
print("¡Hasta la próxima!")
```

Desde la terminal vamos al directorio donde tenemos la carpeta de la práctica que estamos realizando y ejecutamos el siguiente comando y ejecutamos el código:

- `python3.7 prueba.py`

Elementos de un programa

El lenguaje Python está formado por elementos (tokens) de diferentes tipos:

- palabras reservadas (keywords)
- funciones integradas (built-in functions)
- literales
- operadores
- delimitadores
- identificadores

Elementos de un programa

- Líneas y espacios:

Un programa de Python está formado por líneas de texto.

```
radio = 5
area = 3.14159242 * radio ** 2
print(area)
```

Se recomienda que cada línea contenga una única instrucción, aunque puede haber varias instrucciones en una línea, separadas por un punto y coma (;).

```
radio = 5; area = 3.14159242 * radio ** 2
print(area)
```

Por motivos de legibilidad, se recomienda que las líneas no superen los 79 caracteres. Si una instrucción supera esa longitud, se puede dividir en varias líneas usando el carácter contrabarra (\):

```
radio = 5
area = 3.14159265358979323846 \
       * radio ** 2
print(area)
```

Elementos de un programa

Palabras reservadas (keywords)

Las palabras reservadas de Python son las que forman el núcleo del lenguaje Python. Son las siguientes:

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

Estas palabras no pueden utilizarse para nombrar otros elementos (variables, funciones, etc.), aunque pueden aparecer en cadenas de texto.

Elementos de un programa

Literales

Los literales son los datos simples que Python es capaz de manejar:

- números: valores lógicos, enteros, decimales y complejos, en notación decimal, octal o hexadecimal
- cadenas de texto

Operadores

Los operadores son los caracteres que definen operaciones matemáticas (lógicas y aritméticas). Son los siguientes:

```
+       -       *       **      /       //      %       @
<<      >>      &       |       ^       ~
<       >       <=      >=      ==      !=
```

Elementos de un programa

Delimitadores

Los delimitadores son los caracteres que permiten delimitar, separar o representar expresiones. Son los siguientes:

```
'       "       #       \
(       )       [       ]       {       }
,       :       .       ;       @       =       ->
+=      -=      *=      /=      //=     %=      @=
&=      |=      ^=      >>=     <<=     **=
```

Elementos de un programa

Funciones integradas (built-in functions)

Una función es un bloque de instrucciones agrupadas, que permiten reutilizar partes de un programa.

Python incluye las siguientes funciones de forma predeterminada (es decir, estas funciones siempre están disponibles):

```
abs()                dict()     help()     min()                setattr()
          all()                 dir()                hex()              next()
slice()   any()                 divmod()   id()                          object()
sorted()  ascii()       enumerate() input()          oct()
          staticmethod()                             bin()              eval()
          int()         open()    str()              bool()     exec()  isinstance()
          ord()       sum()                        bytearray()          filter()
          issubclass()                             pow()              super()
          bytes()       float() iter()     print()  tuple()    callable()
          format()    len()                property() type()    chr()
          frozenset()                              list()     range()  vars()

classmethod()                   getattr()  locals()  repr()             zip()
          compile()  globals()  map()                reversed() __import__()
```

Elementos de un programa

Funciones adicionales

Un programa puede definir nuevas funciones o redefinir las funciones integradas. Los nombres de las funciones no pueden coincidir con las palabras reservadas.

Un programa puede también importar nuevas funciones que se encuentran definidas en otros ficheros llamados módulos.

Python incluye una biblioteca de módulos (llamada Biblioteca estándar) especializados en todo tipo de tareas.

Además de la biblioteca estándar, existen miles de módulos escritos por diferentes programadores y accesibles en Internet. El principal repositorio de módulos es el Python Package Index (Índice de paquetes de Python), más conocido por PyPI.

Programa básico

Nombres de archivos

Se recomienda que los nombres de los archivos de programas:

- estén escritos en minúsculas
- utilicen guiones bajos para separar palabras o números

Programa en Python

```
def main():
    print("¡Hola, mundo!")


if __name__ == "__main__":
    main()
```

Secuencias

- Tuplas

```
>>> (1, "a", 3.14)
(1, 'a', 3.14)

>>> 1, "a", 3.14
(1, 'a', 3.14)

>>> len((1, "a", 3.14))
3

>>> ()
()

>>> len(())
0
```

En Python, una tupla es un conjunto ordenado e inmutable de elementos del mismo o diferente tipo.

Las tuplas se representan escribiendo los elementos entre paréntesis y separados por comas.

Secuencias

- Rangos

```
>>> lista = [1, "abcde", 45.5, -32]
>>> lista
[1, 'abcde', 45.5, -32]

>>> lista = [ ]
>>> lista
[ ]

>>> x = range(10)
>>> x
range(0, 10)
>>> list(x)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> range(7)
range(0, 7)
>>> list(range(7))
[0, 1, 2, 3, 4, 5, 6]
```

Una **lista** es un conjunto ordenado de elementos del mismo o diferente tipo, cuyo contenido puede modificarse.

Se representan escribiendo los elementos entre corchetes y separados por comas.

Las variables de tipo lista hacen referencia a la lista completa.

El tipo **range** es una lista inmutable de números enteros en sucesión aritmética.

Inmutable significa que, a diferencia de las listas, los range no se pueden modificar.

Una sucesión aritmética es una sucesión en la que la diferencia entre dos términos consecutivos es siempre la misma.

Secuencias

- Listas

```
>>> primos = [2, 3, 5, 7, 11, 13]
>>> diasLaborables = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes"]

>>> directores = [ ["Stanley Kubrick", ["Senderos de Gloria", 1957]], ["Woody Allen", ["Hannah y sus hermanas", 1986]] ]
```

Concatenación

```
>>> vocales = ["E", "I", "O"]
>>> vocales
['E', 'I', 'O']
>>> vocales = vocales + ["U"]
>>> vocales
['E', 'I', 'O', 'U']
```

Las **listas** son conjuntos ordenados de elementos (números, cadenas, listas, etc). Las listas se delimitan por corchetes ([ ]) y los elementos se separan por comas.

Secuencias

- Manipular listas y sublistas

Cada elemento se identifica por su posición en la lista, teniendo en cuenta que se empieza a contar por 0.

```
>>> fecha = [27, "Octubre", 1997]
>>> fecha[0]
27
>>> fecha[1]
Octubre
>>> fecha[2]
1997
```

```
>>> fecha = [27, "Octubre", 1997]
>>> fecha[3]
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    fecha[3]

  Index error: list index out of range
```

```
>>> fecha = [27, "Octubre", 1997]
>>> fecha[2] = 1998
>>> fecha[0]
27
>>> fecha[1]
Octubre
>>> fecha[2]

1998
```

Secuencias

- Manipular listas y sublistas

Cada elemento se identifica por su posición en la lista, teniendo en cuenta que se empieza a contar por 0.

```
>>> dias = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"]
>>> dias[1:4] # Se extrae una lista con los valores 1, 2 y 3
['Martes', 'Miércoles', 'Jueves']
>>> dias[4:5] # Se extrae una lista con el valor 4
['Viernes']
>>> dias[4:4] # Se extrae una lista vacía
[]
>>> dias[:4]  # Se extrae una lista hasta el valor 4 (no incluido)
['Lunes', 'Martes', 'Miércoles', 'Jueves']
>>> dias[4:]  # Se extrae una lista desde el valor 4 (incluido)
['Viernes', 'Sábado', 'Domingo']
>>> dias[:]   # Se extrae una lista con todos los valores
['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado', 'Domingo']
```

Estructuras de control

- Secuencias condicionales

  if ... elif ... else

```
edad = int(input("¿Cuántos años tiene? "))
if edad < 0:
    print("No se puede tener una edad negativa")
elif edad < 18:
    print("Es usted menor de edad")
else:
        print("Es usted mayor de edad")
```

Estructuras de control

- Iteraciones

  bucle for

```
print("Comienzo")
for i in [3, 4, 5]:
    print(f"Hola. Ahora i vale {i} y su cuadrado {i ** 2}")
print("Final")
```

```
print("Comienzo")
for i in range(10):
    print("Hola ", end="")
print()
print("Final")
```

Estructuras de control

- Iteraciones

  bucle while

```
numero = int(input("Escriba un número positivo: "))
while numero < 0:
    print("¡Ha escrito un número negativo! Inténtelo de nuevo")
    numero = int(input("Escriba un número positivo: "))
print("Gracias por su colaboración")
```

Funciones

```
def licencia():
    print("Copyright 2020 Bartolomé Sintes Marco")
    print("Licencia CC-BY-SA 4.0")
    return


print("Este programa no hace nada interesante.")
licencia()
print("Programa terminado.")
```

```
Este programa no hace nada interesante.
Copyright 2020 Bartolomé Sintes Marco
Licencia CC-BY-SA 4.0

Programa terminado.
```

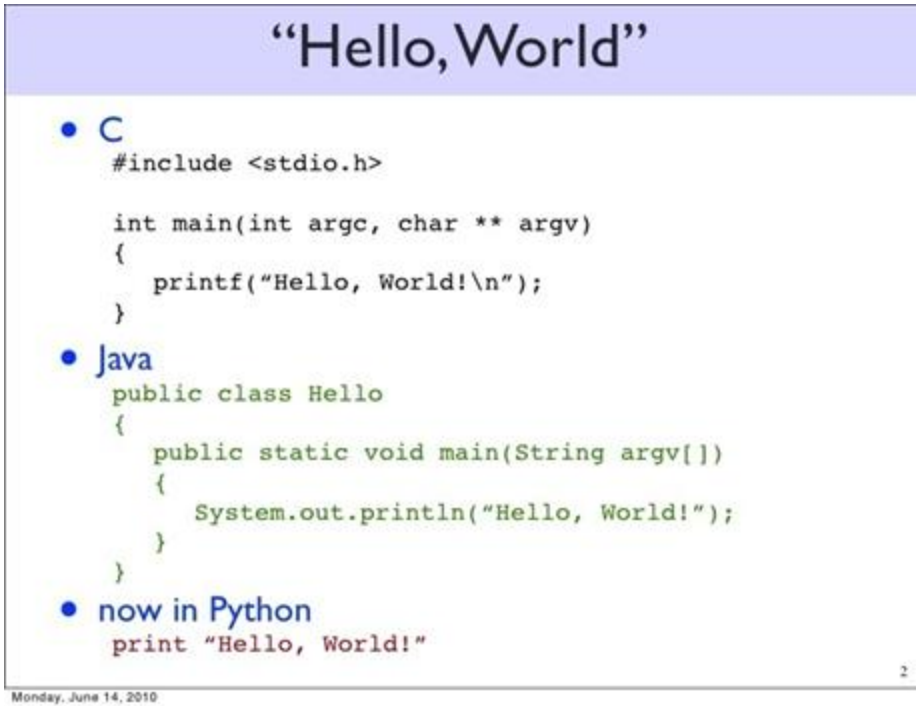
```
print("Este programa no hace nada interesante.")
licencia()
print("Programa terminado.")

def licencia():
    print("Copyright 2020 Bartolomé Sintes Marco")
    print("Licencia CC-BY-SA 4.0")
    return
```

```
Este programa no hace nada interesante.
Traceback (most recent call last):
  File "ejemplo.py", line 2, in <module>
    licencia()

NameError: name 'licencia' is not defined
```

Ahora a programar



Casos prácticos

Variables y operaciones aritméticas

- **Prog1**: Escriba un programa que pida dos números y que escriba su media aritmética.
  - Se recuerda que la media aritmética de dos números es la suma de ambos números dividida por 2.
- **Prog2**: Escriba un programa que pida el peso (en kilogramos) y la altura (en metros) de una persona y que calcule su índice de masa corporal (imc).
  - Se recuerda que el imc se calcula con la fórmula imc = peso / altura^2

Estructura condicional

- **Prog3**: Escriba un programa que pida primero un número par y luego un número impar (positivos o negativos). En caso de que uno o los dos valores no sea correcto, se mostrará un único aviso.
- **Prog4**: Escriba un programa que pida primero un número par (positivo o negativo) y si el valor no es correcto, muestre un aviso. Si el valor es correcto, pedirá un número impar (positivo o negativo) y si el valor no es correcto, mostrará un aviso.

Casos prácticos

Bucles

- **Prog5:** Escriba un programa que pida dos números enteros y escriba qué números son pares y cuáles impares desde el primero hasta el segundo.
  - Un número es par cuando el resto de su división entre dos es cero (numero % 2 == 0) e impar cuando no lo es.
- **Prog6**: Escriba un programa que pida un número entero mayor que cero y que escriba sus divisores.
  - Un número es divisible por otro cuando el resto de su división es cero (numero % divisor == 0).
  - Se puede hacer un programa más rápido, teniendo en cuenta que los divisores son siempre menores o iguales que la mitad del número (salvo el propio número, que es divisor de sí mismo). Es decir, no hace falta probar todos los números entre 1 y el propio número, sino únicamente hasta la mitad. Si se hace así, no hay que olvidarse de añadir el propio número a la lista de divisores.

Casos prácticos

Listas

- **Prog7:** Escriba un programa que permita crear una lista de palabras. Para ello, el programa tiene que pedir un número y luego solicitar ese número de palabras para crear la lista. Por último, el programa tiene que escribir la lista.
- **Prog8**: Escriba un programa que permita crear una lista de palabras y que, a continuación, pida una palabra y diga cuántas veces aparece esa palabra en la lista.

Casos prácticos

Funciones

- **Prog8:** Implementar alguno de los programas anteriores en funciones y organizar el código.
  - Se propone el Prog5 y Prog6